You said:

Williamson __ CS1 ...
Document

Final Project_ 2 - D...
PDF

Final Project Requi...
PDF

I am looking to build a website using Drafter as a project for a college-level Computer Science course. The course focuses on learning and building Python programming skills, and the project interweaves this with a close up on the Drafter python library, which has its own specific functionalities and is incorporated to assist in the creation of the website. My code is to be compiled in a single place, in so that it could be run on a platform such as Thonny without error and can be copied from an online repository such as Github in a functional state. Detailed instructions/ rubrics will be attached with this prompt. Additionally, an attached docx file "Williamson // CS1 Final Project 1: Design" gives detail as to what I want to create. Please look through it in its entirety to gleam important insight into what I want to build.

My initial idea was to create a Wiki-style website that documented information on a given topic, likely a video game of my choosing. However, I think it would be neat to instead create an adaptive website that will assist in creating and maintaining a personalized Wiki-style library of information that the user can interact with and put details into. For instance, a user could define a term, create a new page specifically for that term and its definition, and in its definition it could include other key terms that could be given their own definitions and pages (and accessed by an in-line link). Additionally, once terms are created, they would be easily accessible through the search feature. I really enjoy this concept because of how free from limitations it is-- it could be used for myriad purposes, from cataloging one's personal life experiences to documenting information on a piece of media to keeping track of a friend group's inside jokes.

With the attached project description instructions/ rubric, please look through for details about what must be included in the program, then assess whether the concept for my website could feasibly meet this specs.

If meeting these specs is explicitly undoable based on the scope I have for the project, suggest alterations or places to expand.

Otherwise, write a prototype program to fulfill the aforementioned concept for the website. If I ever refer to the website as a "video game wiki", disregard it, as that was an earlier idea in development-- I want to create an open-ended Wiki builder. Ensure you are strictly following Drafter conventions from the Python library. Ask for clarification if anything is unclear.

Worked for 10m 4s >

ChatGPT said:

Your open-ended wiki builder is feasible within the course requirements. I built a single-file Drafter prototype that implements the concept.

The package contains `main.py`, setup instructions, a detailed rubric checklist, and your original planning documents.

The prototype supports:

- Creating, editing, categorizing, searching, and deleting terms.
- Inline references written as `[[another term]]`.
- Opening a creation form when a linked term doesn't exist.
- Updating bracketed references when a term is renamed.
- Customizing the wiki name.
- Backing up and restoring the collection.

Programming requirement	Implementation
At least 7 routes	15 routes
At least 5 pages	9 distinct page views
At least 4 changing State fields	7 fields
Second dataclass and collection	`WikiArticle` and `list[WikiArticle]`
At least 3 input fields	Multiple fields using all four listed input types
Meaningful conditionals and loops	Validation, search, article lookup, and reference updates
Tests, annotations, and documentation	64 assertions covering every route; typed functions and docstrings

Two adjustments address gaps in the original design: actual input components satisfy the input requirement, and distinct functional views satisfy the page requirement without needing separately coded pages for every article.

Storage is the main limitation. Drafter's ordinary state resets on reload, so this version provides explicit copy-and-paste backups. Automatic saving or shared accounts would require additional work.

Drafter Docume...

To run it, extract the ZIP, install `drafter==2.2.1` in Thonny's package manager, and run `main.py`. All application code and tests are in that file.

The tests, Drafter page verification, and deployment compilation passed. Browser click-through testing could not be completed here, so test locally and after publishing. Before submission, also upload your **complete AI conversation**, required planning worksheets, and demonstration video; the README explains these remaining requirements.

Reviewed source documents:

You said:

I have Thonny set up properly. Where is the file to unzip and place in Thonny? I do not see it in your response.

ChatGPT said:

Here's the download: personal_wiki_builder.zip.

1. Download it, then right-click the ZIP and select **Extract All**.
2. In Thonny, select **File** → **Open** and open **main.py** from the extracted folder.
3. Click **Run** or press **F5**.

You said:



Pasted text(5).txt
Document

How do I use this? I am currently finding the category, definition or description, and bottom unlabeled text box on the Create a term section a bit confusing to operate. If it helps, these are the current looks of the sections. Pasted below is the current code that I am

working with. Feel free to make targeted changes to the code to improve functionality.

Worked for 1m 25s >

ChatGPT said:

The **bottom box is the category input**. Its label appeared beside the large description box, making it look unlabeled. That layout was confusing.

Here's how the fields work:

Field	What to enter	Example
Term title	The name of the page	`Study group`
Definition or description	The page's actual content	`Our weekly meeting to review [[Calculus]].`
Category	An optional grouping for related pages	`College life`

Click **Save term** after filling them in. Categories are created automatically from what you type; reuse the same category name to group pages together. Leaving it blank uses **Uncategorized**.

The **category dropdown near Search** only filters search results. It doesn't assign a category to the term you're creating.

I made targeted improvements:

- Put each label directly above its input.
- Numbered the three editor fields and added explanations.
- Widened the description and backup text boxes.
- Fixed old validation messages lingering on Home, Settings, Backup, and Help.
- Separated instructional paragraphs for readability.

Download the updated ‹/› **main.py** — no unzipping needed.

Before restarting your current site, use Backup to save your existing entries. Then open this replacement file in Thonny, run it, and restore your backup.

The existing tests and added checks pass, along with Drafter's component verification and deployment compilation.

You said:



I like this improved version a lot more, and I am understanding how to use it. Please keep this overall workflow unchanged. However, after creating a term or when clicking on a term to edit it, I've noticed that the formatting is a bit odd, with new lines splicing information up and underline characters floating under spaces near inline links, as pictured in the screenshot. Note that, as far as I can tell, this only appears in this messy format when clicking on a term-- the search feature (and remainder of the site, as best I can tell) are all otherwise formatted intuitively. If possible, can you update this formatting so that there are not new lines splicing the information up (unless the new lines are a part of the description for the term), and have the information in an easily legible format?

Worked for 1m 9s

ChatGPT said:

Fixed the article formatting while keeping your workflow unchanged.

Download the updated main.py

The update removes generated line breaks and indentation around inline links, eliminates the stray underlined spaces, and preserves line breaks and blank lines you entered in the description.

Tests, article rendering checks, and deployment compilation pass.

Back up your current entries before restarting, then replace in Thonny, run it, and restore your backup.